



CONCEPTS OF SOFTWARE ENGINEERING

Software Modeling



Abdulfattah Alfarra



Outline

- Model
- Agile Model
- UML
- Use Case
- Class Responsibility Collaborator (CRC)
- Class Diagram
- Sequence Diagram



Model

- A **model** is a concept that describes one or more aspects of a problem or a potential solution addressing a problem.
- Traditionally, models are thought of as zero or more diagrams plus any corresponding documentation.



Model

- There are two primary reasons why we model:
 - 1) To understand what it is you are building or
 - 2) To aid your communication efforts within your team or with your project stakeholders.

Agile Model

- From the point of view of simplicity, one could say that system development doesn't need more than coding, testing and listening. If those activities are performed well, the result should always be a system that works.
- In practice, this will not work. One can come a long way without designing but at a given time one will get stuck. The system becomes too complex and the dependencies within the system cease to be clear.



Agile Model

- One can avoid this by creating a design structure (i.e. model) that organizes the logic in the system. Good design will avoid lots of dependencies within a system; this means that changing one part of the system will not affect other parts of the system.
- 



XP Model

- Modeling and documentation are important aspects of XP, just like they are important aspects of any other software development methodology.
- However, XP explicitly advises you to minimize the effort that you invest in these activities to be **just enough**.
- In XP modeling usually used for software design.



Simple design

- **Simple design**—The system should be designed as simply as possible at any given moment. Extra complexity is removed as soon as it is discovered.

Simple design

- The team does not spend very much time looking ahead to future requirements and needs. Nor does it try to build in today the infrastructure to support the features that may be needed tomorrow. Rather, the team focuses on the *current structure of the system, making it as good* as it can be.

XP model

- In XP modeling, we often develop **small, simple** diagrams that focus on one aspect of your software, such as the life cycle of a class or the flow between screens. To do that **UML** diagram can be used.

The Unified Modelling Language

- The Unified Modeling Language (UML) is an industry standard for modeling object-oriented systems.
- It describes notations for a number of different models that may be produced during OO analysis and design.
- UML is now a *de facto* standard for OO modelling.

What is UML?

- UML is a Language for **Specifying, Visualizing, Constructing, and Documenting** Software Artifacts
- What Does a Modeling Language Provide?
 - ***Model Elements***: Concepts and Semantics
 - ***Notation***: Visual representation of Model Elements
 - ***Guidelines***: Hints and Suggestions for Using Elements in Notation.

Types Design models

- Design models show the objects and object classes and relationships between these entities.
- *Static models* describe the static structure of the system in terms of object classes and relationships.
- *Dynamic models* describe the dynamic interactions between objects.



Use case

- **Use cases** serve the purpose to capture specific user requirements in terms of interactions between the user and the system.

Use case

- A use case has been described as “a generalized description of a set of interactions between the **system** and one or more **actors**, where an actor is either a user or another system”
- The use-case model helps us ensure that we have included the right features for the stakeholders and have not forgotten anything, and allows us to easily see the whole system.



Use case

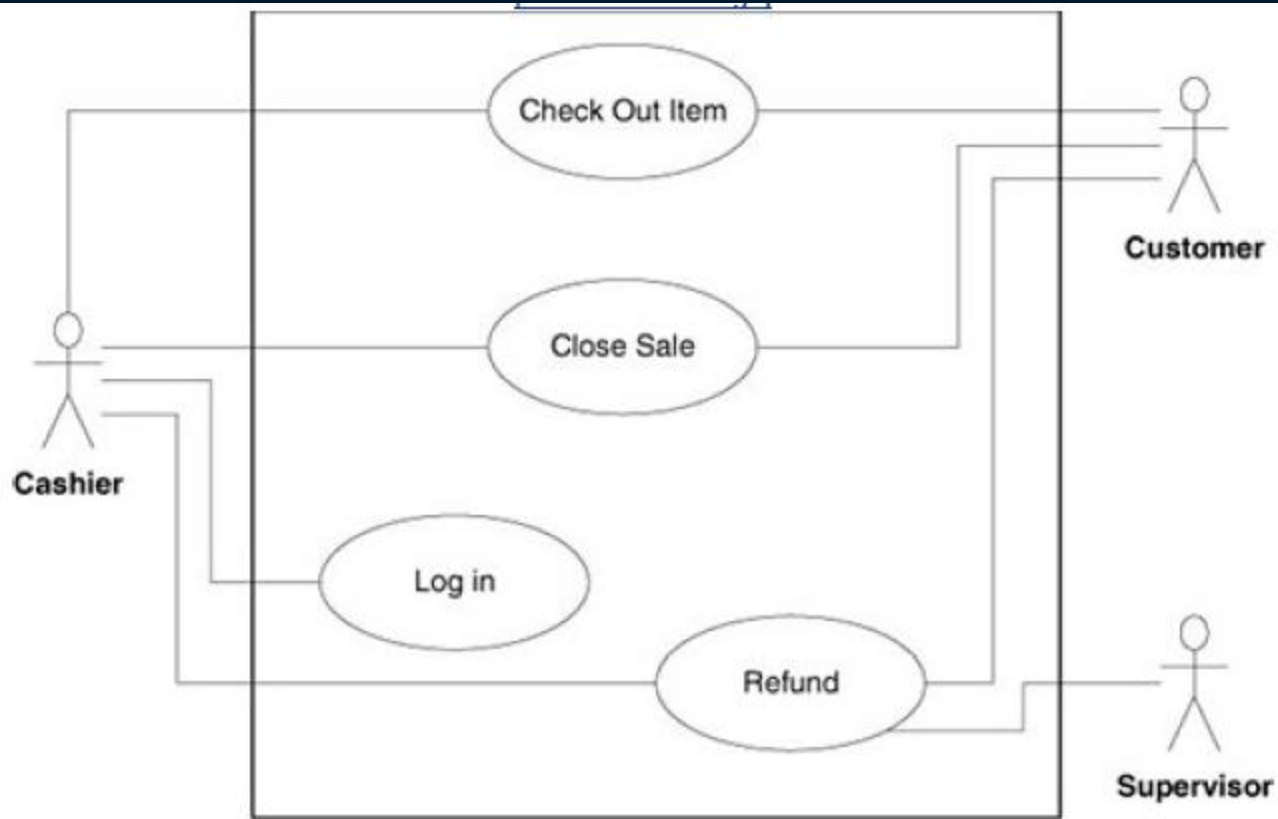
Use case detail level

- *High Level* - Brief with no details
- *Expanded* - More detailed with information about every step in the process. Don't describe how the system responds.

Use Cases Process

- Use Case consist of the following steps:
 - **Define system boundary.** Identify actors and use cases
 - Draw **high-level** use-case diagram
 - Write expanded use cases for the most critical, influential or risky use cases. postpone rest use cases.

Use Cases Example



Use Cases Process

- **Boundary diagram:** The large rectangle is the system boundary. Everything inside the rectangle is part of the system under development.

Use Cases Process

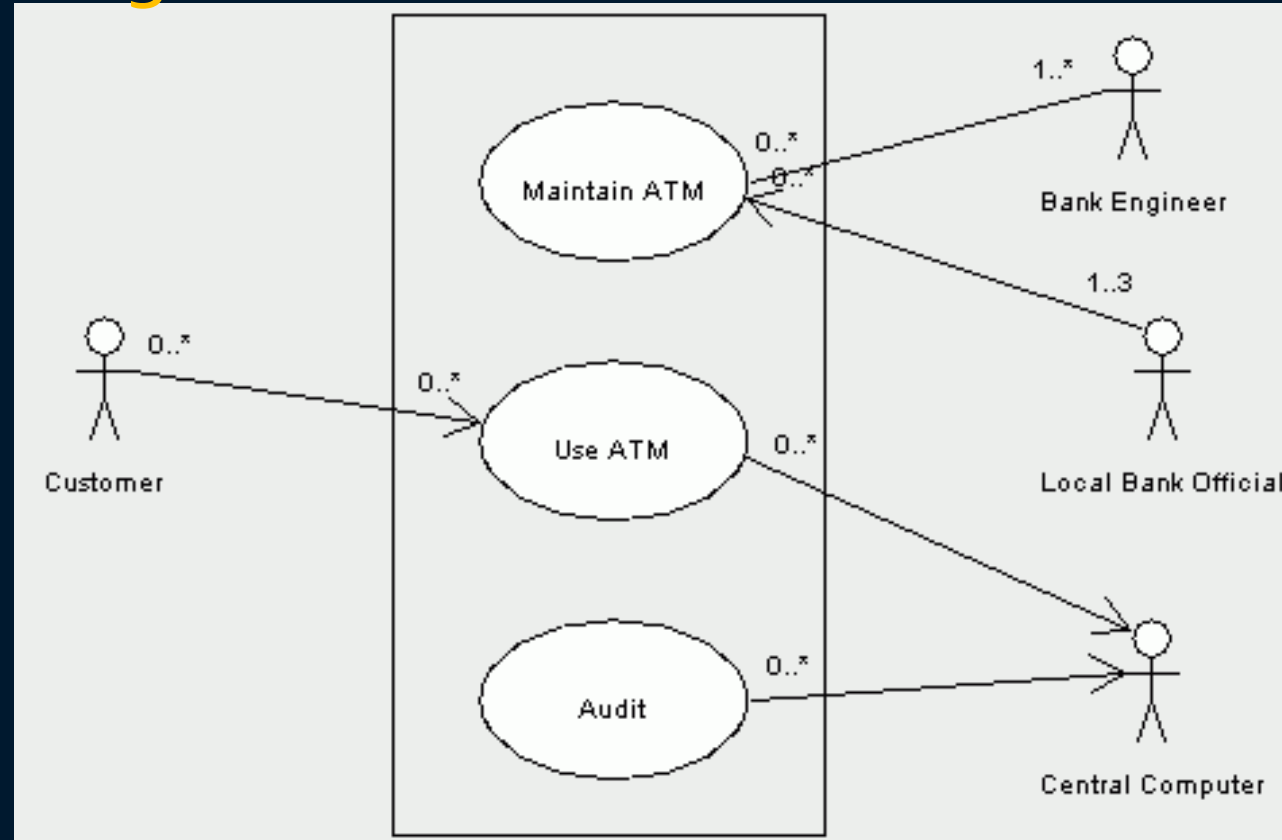
- Outside the rectangle are the *actors* that act on the system. Actors are entities outside the system and provide the motivation for the system. Typically, actors are human users (in the example: Cashier, customer and supervisor).
- They might also be other systems or even devices, such as real-time clocks.

Use Cases Process

- Inside the boundary rectangle are the **use cases**: the ovals with names inside. For each actor, use cases identify the processes they initiate or participate in (In the example: Check out item, close sale, login, Refund).
- The **lines** connect the actors to the use cases they stimulate. Avoid using arrows; nobody really knows what the direction of the arrowheads means.

Use case: Another example

High Level Use Case

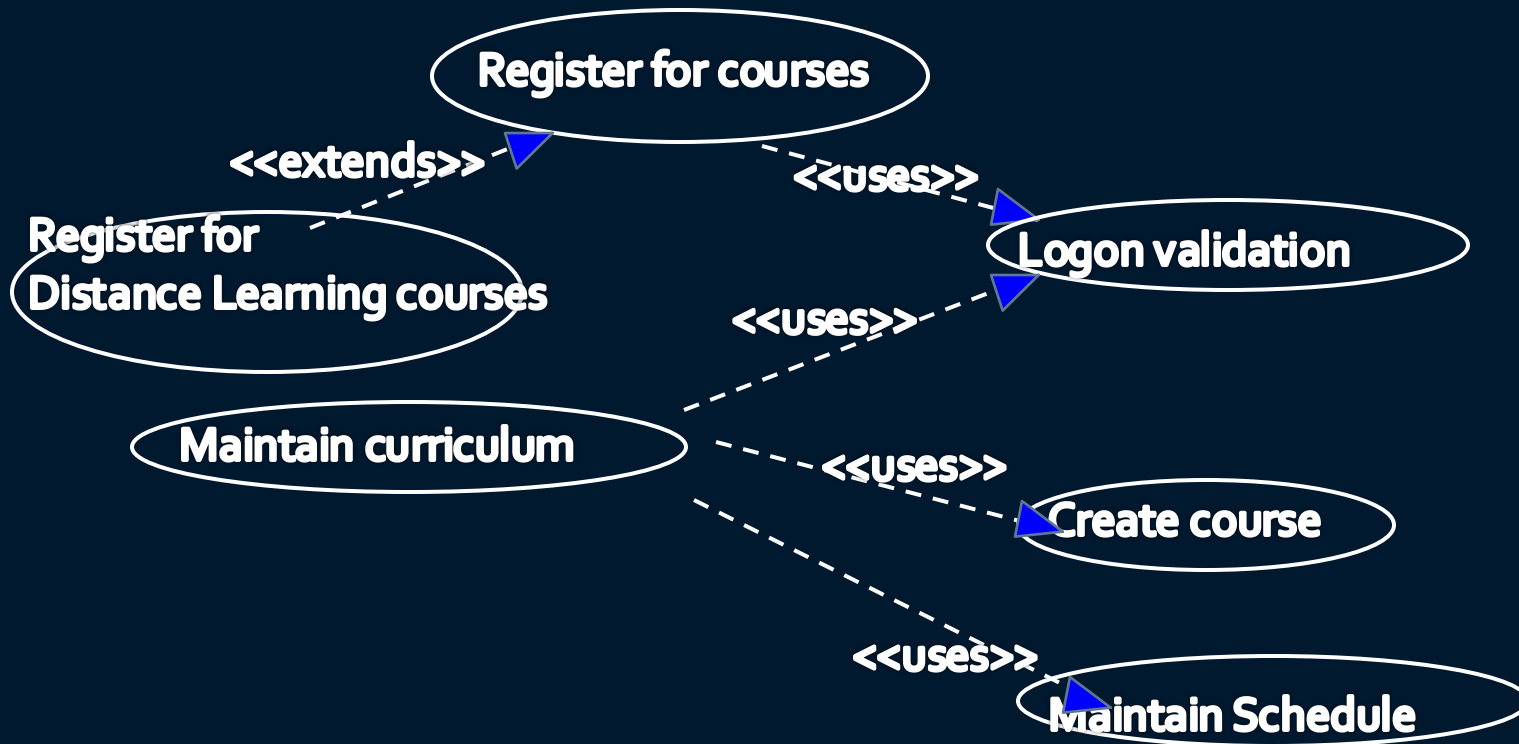


Use case

Uses and Extends in Use Case Diagram

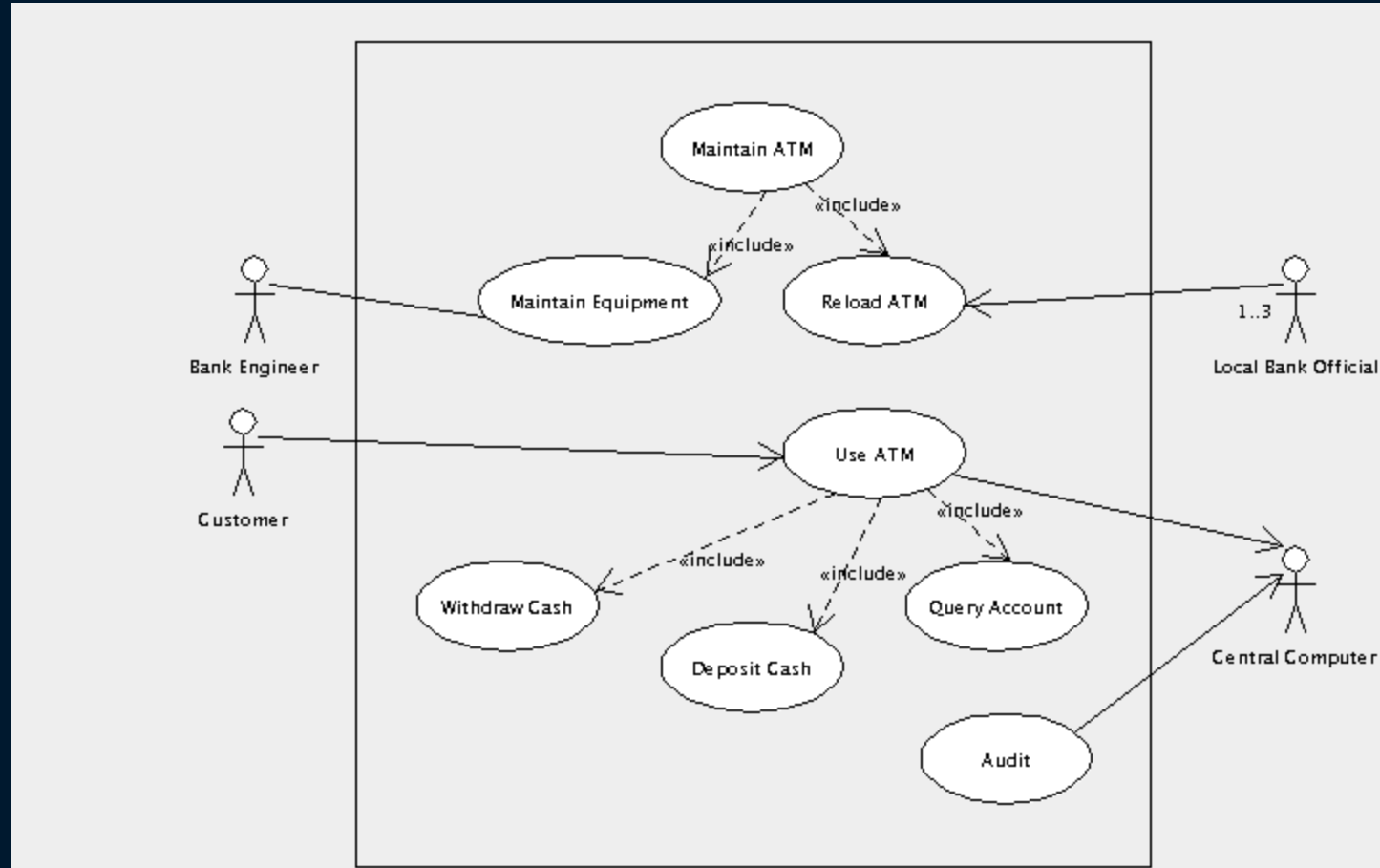
A uses relationship shows behavior common to one or more use cases

An extends relationship shows optional/exceptional behavior



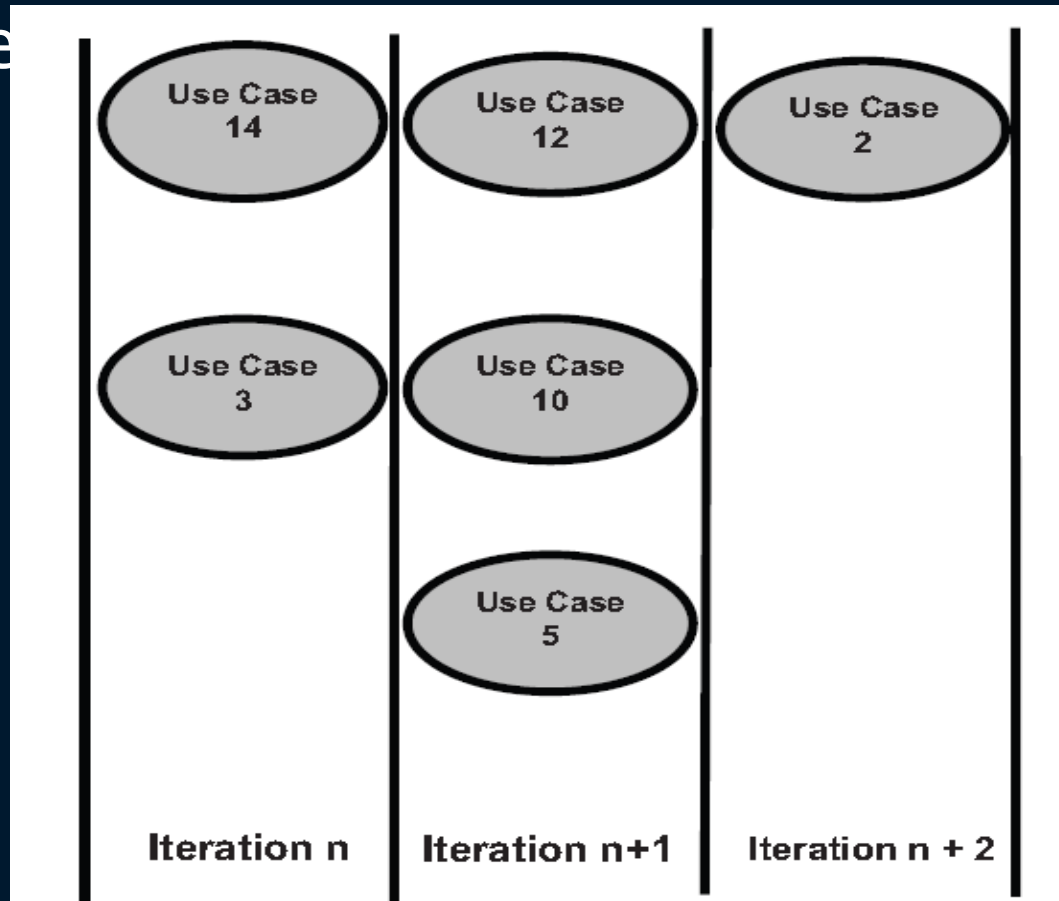
Use case

Expanded Use Case

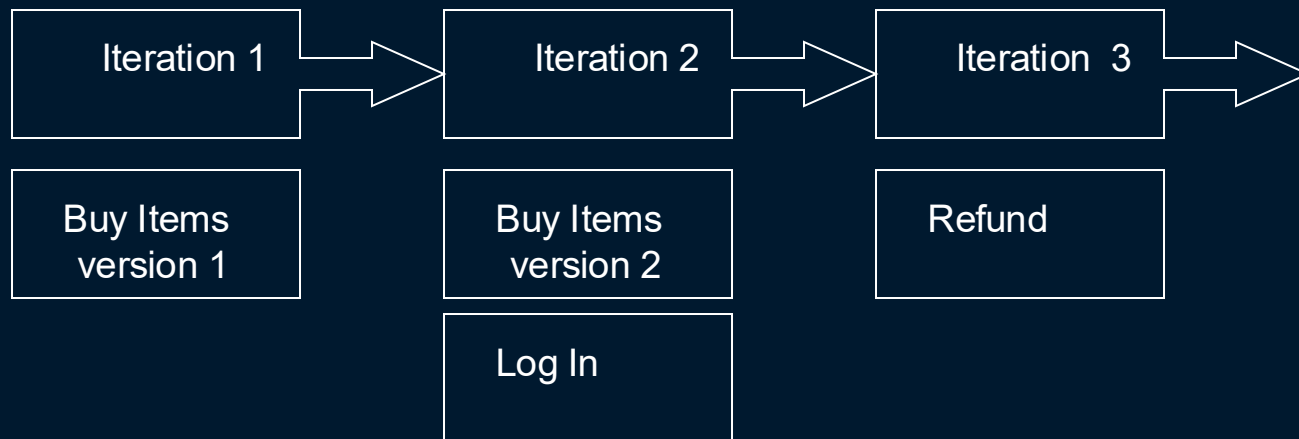


Allocation of use cases to Iterations

- Use cases are used to plan content of iterations



Allocation of use cases to Iterations



Use cases and user stories

- Use cases and stories are similar in that they are both ways to organize requirements.
- They are different in that they organize for different purposes.
- **Use cases** organize requirements to form a narrative of how users relate to and use a system. Hence they focus on **user goals** and how **interacting with a system** satisfies the goals.

User stories and use cases

User Stories

- Provide a small-scale and easy-to-use presentation of information. Are generally formulated in the everyday language of the user and contain little detail, thus remaining open to **interpretation**. They should help the reader understand what the software should accomplish.
- Must be accompanied by acceptance testing procedures (acceptance criteria) for clarification of behavior where stories appear ambiguous.

User stories and use cases

Use Cases

- Describe a process and its steps in detail, and may be worded in terms of a formal model. A use case is intended to provide sufficient detail for it to be understood on its own. A use case has been described as “a generalized description of a set of interactions between the system and one or more actors, where an actor is either a user or another system”.
- May be delivered in a stand-alone document



Use cases and user stories

- XP stories (and similar things, often called features) break requirements into chunks for planning purposes. Stories are explicitly broken down until they can be estimated as part of XP's release planning process.
 - User stories quite simple and more straightforward than use cases.
- 



The Class Diagram

- Class diagrams are useful in all forms of object-oriented programming (OOP)
 - UML class diagrams allow us to denote the static contents of classes and the relationships between classes.
 - It provides a wide variety of usages; from modeling the domain-specific data structure to detailed design of the target system.
- 



Class Responsibility Collaborator

- A Class Responsibility Collaborator (CRC) model is part of XP modeling which has many shared characteristics with class diagram.
- a CRC model, which generally comes before a class diagram because it is detailed than class diagrams.

A Class Responsibility Collaborator (CRC)

- A Class Responsibility Collaborator (CRC) model is a collection of standard index cards that have been divided into three sections. A **class** represents a collection of similar objects, a **responsibility** is something that a class knows or does, and a **collaborator** is another class that a class interacts with to fulfill its responsibilities.

A Class Responsibility Collaborator (CRC)

Class Name	
Responsibilities	Collaborators

A Class Responsibility Collaborator (CRC)

Student	
Student number Name Address Phone number Enroll in a seminar Drop a seminar Request transcripts	Seminar



CRC

- A **class** represents a collection of similar objects. An object is a person, place, thing, event, concept, screen, or report that is relevant to the system at hand.
- For example, Figure below shows a shipping/inventory control system with the classes such as **Inventory Item, Order, Order Item, Customer, and Surface Address.**

CRC Example

Inventory Item	
Item number Name Description Unit Price Give price	

Order	
Order number Date ordered Date shipped Order items Calculate order total Print invoice Cancel	Order Item Customer

Order Item	
Quantity Inventory item Calculate total	Inventory item

CRC Example -Continue

Customer	
Name	Order Surface Address
Phone number	
Customer number	
Make order	
Cancel order	
Make payment	

Surface Address	
Street	
City	
State	
Zip	
Print label	

Figure 2. A CRC model for a simple shipping/inventory control system.

CRC

- The name of the class appears across the top of the card.
- **Class Name** creates the vocabulary of our analysis
 - Use **nouns** as class names, think of them as simple agents
 - **Verbs** can also be made into nouns, if they are maintain state
 - E.g., “reads card” suggests **CardReader**,
- Use pronounceable names:
 - If you cannot read aloud, it is not a good name
- Use capitalization to initialize Class names
 - E.g., CardReader
- Avoid obscure, ambiguous abbreviations
 - E.g., is TermProcess something that terminates or something that runs on a terminal?
- Try *not* to use digits within a name, such as CardReader1



CRC

- A **responsibility** is anything that a class knows or does. For example, customers have names, customer numbers, and phone numbers.
- These are the things that a customer knows. Customers also order products, cancel orders, and make payments



CRC

- Describes a class's **behaviors**
 - Describe *what* is to be done, not *how*!
- Use short verb phrases
 - E.g.: “reads card” or “look up words”
- How do constraints of index cards guide class analysis?
 - A good measure of appropriate complexity
 - If you cannot fit enough tasks on a card, maybe you need to divide tasks between classes, on different cards?

CRC

- **Collaborators** : The names of other classes with which the class will collaborate to fulfill its responsibilities.
- Lists important **suppliers** and possibly clients of a class
- Why are classes that supply services more important here?
 - Suppliers are necessary for the description of responsibilities
- As you write down responsibilities for a class, add any suppliers needed for them
 - For example “read dictionary” obviously implies that a “dictionary” as a collaborator.

Class Diagram



A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics.

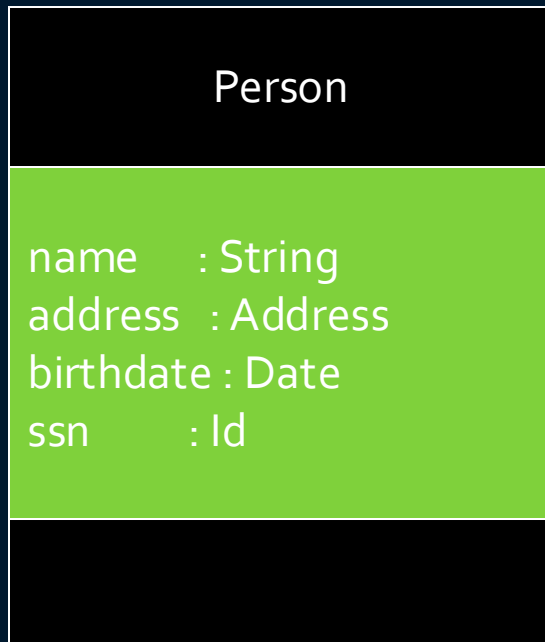
Graphically, a class is turned into a rectangle, usually including its name, attributes, and operations in separate, designated compartments.

Class Diagram



The **name of the class** is the only required tag in the graphical representation of a class. It always appears in the top-most section.

Class Diagram



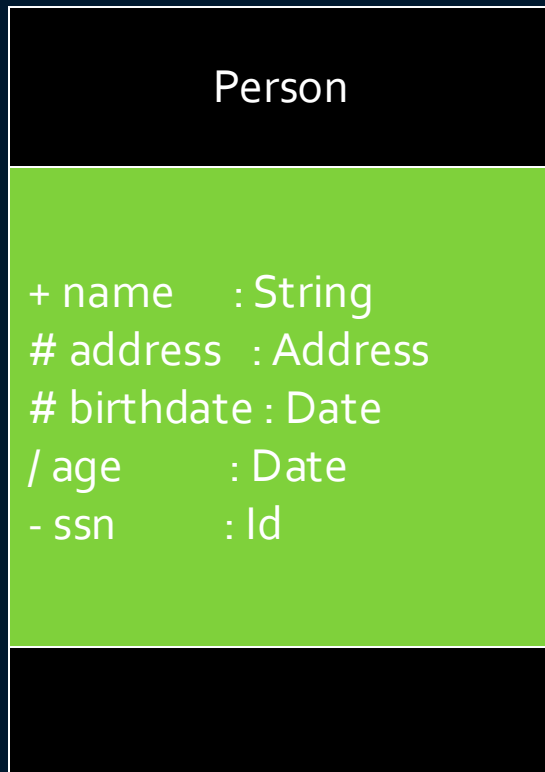
An *attribute* is a named property of a class that describes the object being modeled.

In the class diagram, attributes appear in the second compartment just below the section.

Attributes are usually listed in the form:

attributeName : Type

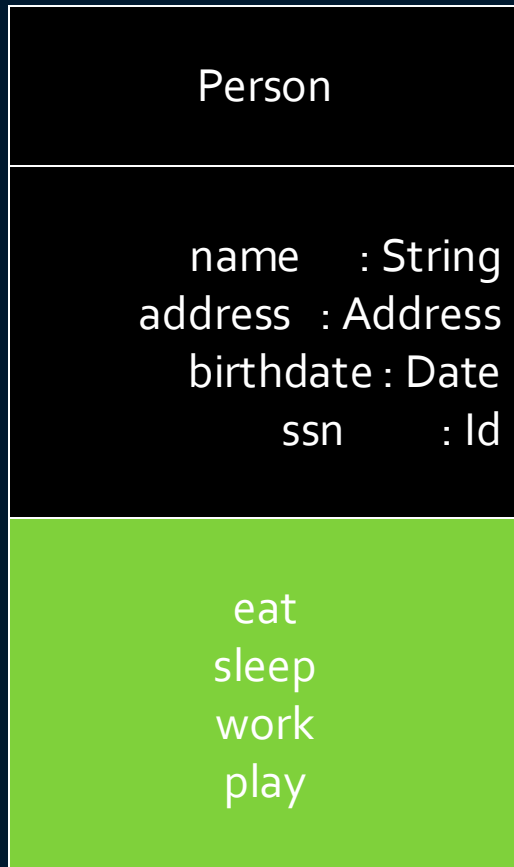
Class Diagram



Attributes can be:

- + public
- # protected
- private
- / derived

Class Diagram



Operations describe the class behavior and appear in the third compartment.

Class Diagram

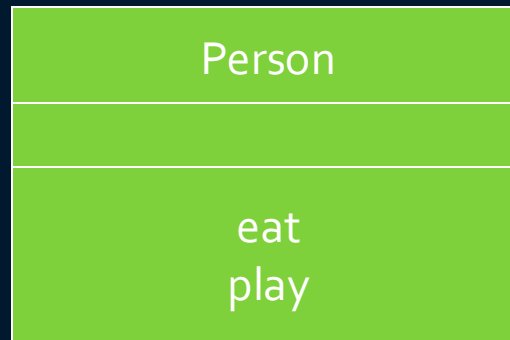
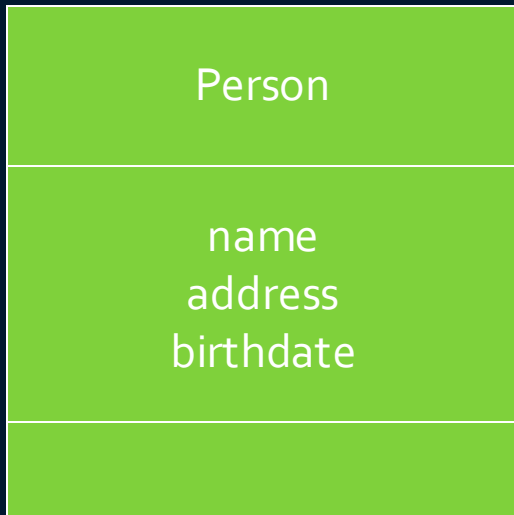
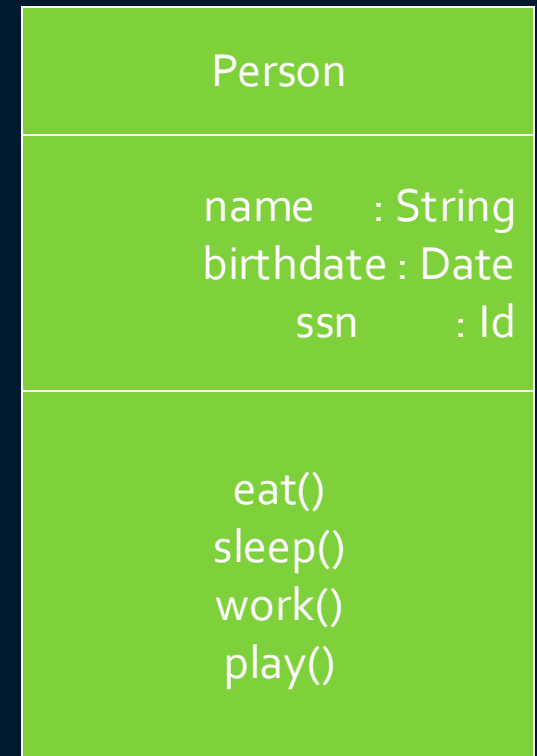
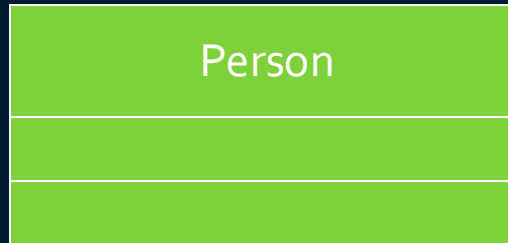
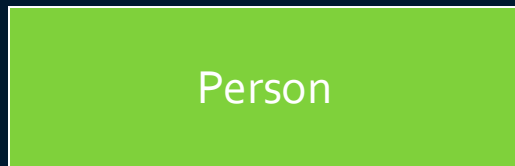
PhoneBook

```
newEntry (n : Name, a : Address, p : PhoneNumber, d : Description)  
getPhone ( n : Name, a : Address) : PhoneNumber
```

You can specify an operation by stating its signature: listing the name, type, and default value of all parameters, and, in the case of functions, a return type.

Class Diagram

When drawing a class, you needn't show attributes and operation in every diagram.





Class Diagram

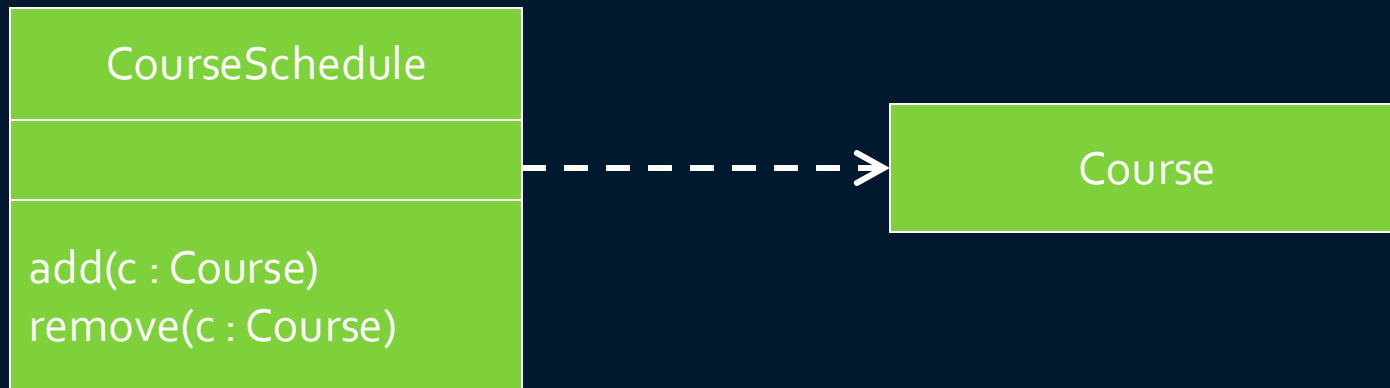
In UML, object interconnections are modeled as **relationships**.

There are three kinds of relationships in UML:

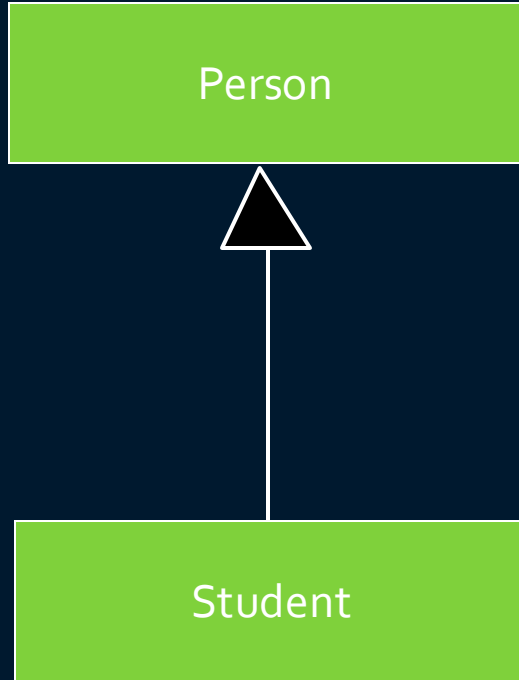
- dependencies
 - generalizations
 - associations
- 

Class Diagram

A *dependency* indicates a semantic relationship between two or more elements. The dependency from *CourseSchedule* to *Course* exists because *Course* is used in both the **add** and **remove** operations of *CourseSchedule*.



Class Diagram



A *generalization* connects a subclass to its superclass. It denotes an inheritance of attributes and behavior from the superclass to the subclass and indicates a specialization in the subclass of the more general superclass.

Class Diagram

If two classes in a model need to communicate with each other, there must be link between them.

An *association* denotes that link.



Class Diagram

We can indicate the *multiplicity* of an association by adding *multiplicity adornments* to the line denoting the association.

The example indicates that a *Student* has one or more *Instructors*:



Class Diagram

The example indicates that every *Instructor* has one or more *Students*:



Class Diagram

We can also indicate the behavior of an object in an association (*i.e.*, the *role* of an object) using *rolenames*.



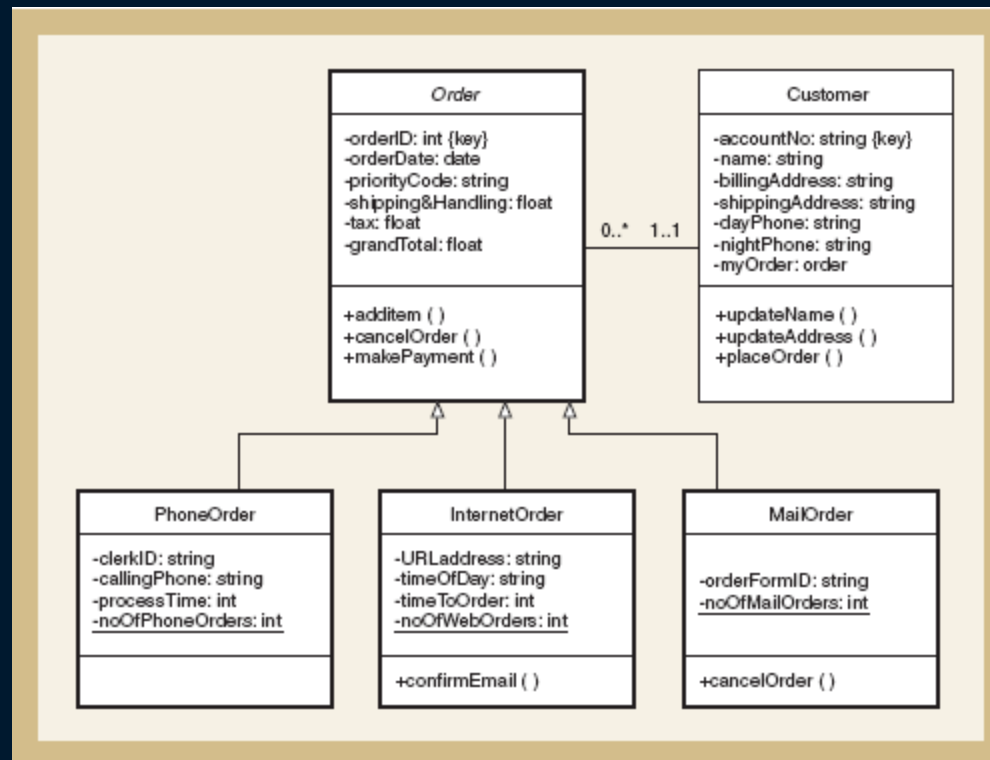
Class Diagram

A *composition* indicates a strong ownership and coincident lifetime of parts by the whole (*i.e.*, they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.



Class Diagram Example

Sample Class Diagram with Design Classes and Inheritance



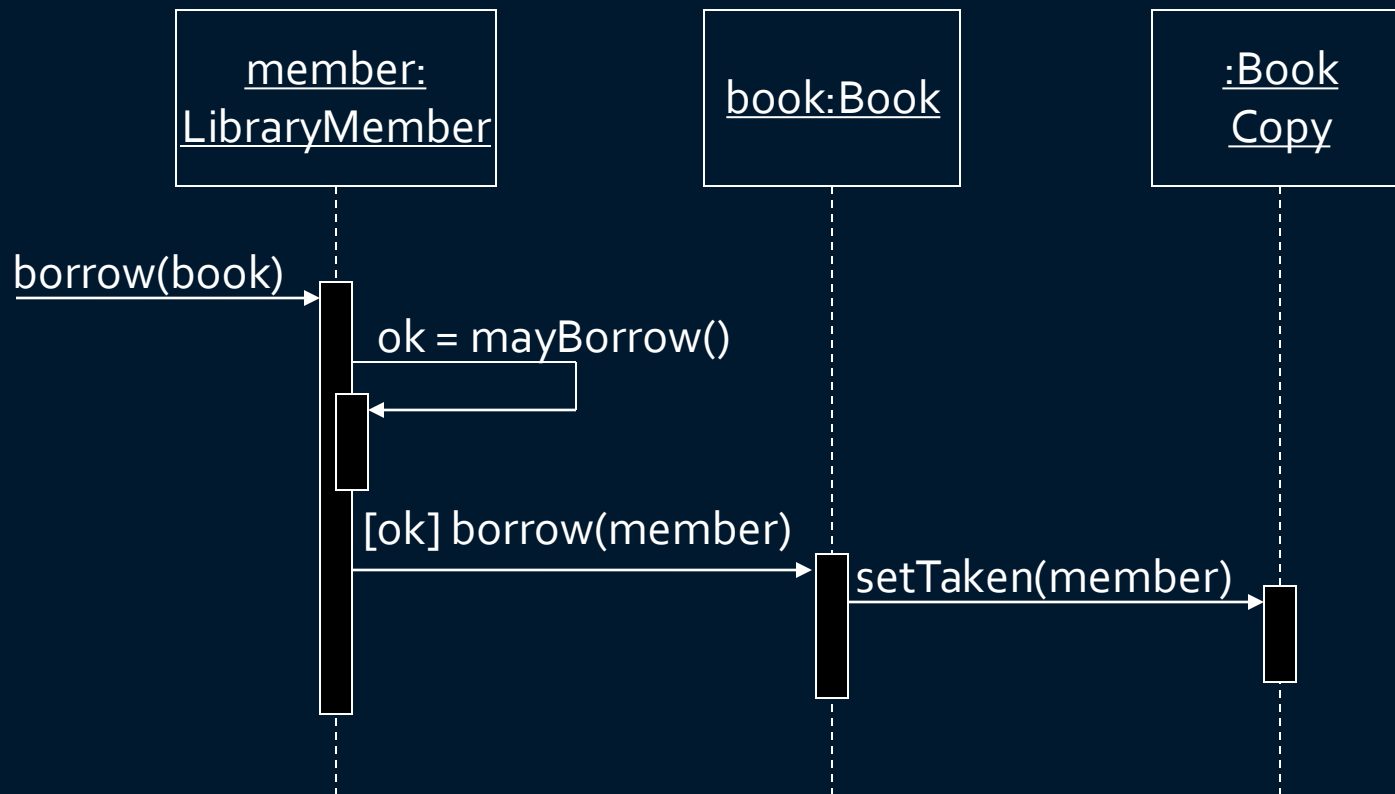
Sequence Diagrams

- Illustrates how objects interact with each other.
- Emphasizes time ordering of messages.
- Can model simple sequential flow, branching, iteration, recursion and concurrency.

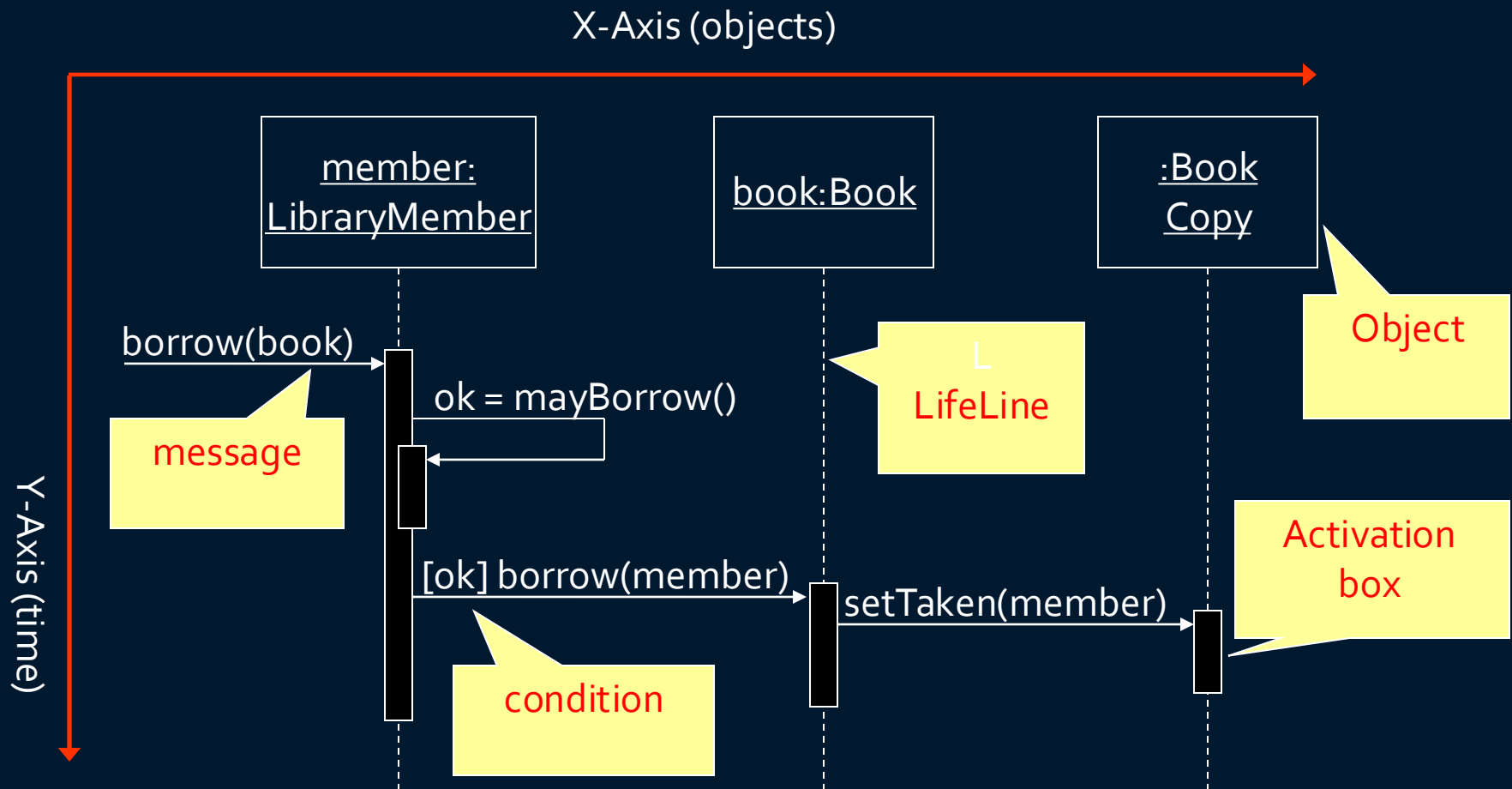
Sequence Diagrams

- In **XP** , Sequence diagrams are a great way to understand and to explain to other people how the objects in a system inter relate as a **function of time**. The utility of sequence diagrams is incredible for at least three reasons:
 - 1) Allows developers to really understand the time-sequenced interaction between system objects.
 - 2) Allows technically oriented customers to get a better understanding of how the system will function.
 - 3) Allows developers to identify “failure points” and facilitates setting up test cases.

A Sequence Diagram Example



A Sequence Diagram

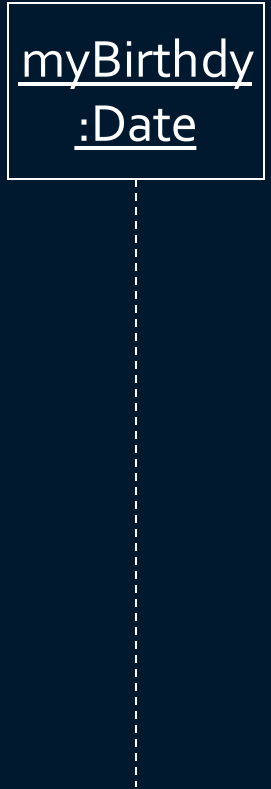


Sequence Diagram

- Sequence models show the sequence of object interactions that take place
 - Objects are arranged horizontally across the top;
 - Time is represented vertically so models are read top to bottom;
 - Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction;
 - A thin rectangle in an object lifeline called activation Box represents the time when the object is the controlling object in the system

Object

- Object naming:
 - syntax: *[instanceName][:className]*
 - Name classes consistently with your class diagram (same classes).
 - Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.
- The *Life-Line* represents the object's life during the interaction

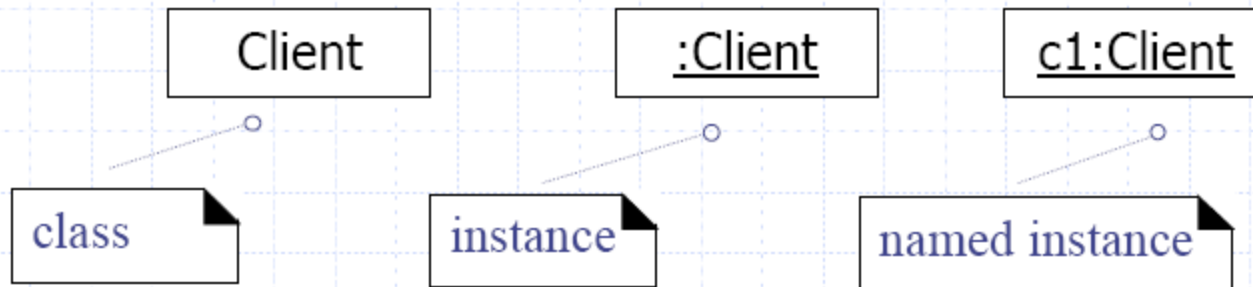


A diagram illustrating UML object notation. It consists of a rectangular box divided into two horizontal compartments. The top compartment contains the text myBirthdy, and the bottom compartment contains the text :Date. A vertical dashed line extends downwards from the bottom of the box.

```
myBirthdy  
:Date
```


Object

- **Classes and Instances**





Messages

- An interaction between two objects is performed as a message sent from one object to another (simple operation call, Signaling, RPC)
 - If object obj_1 sends a message to another object obj_2 some link must exist between those two objects (dependency, same objects)
- 

Messages



- A message is represented by an arrow between the life lines of two objects.
 - Self calls are also allowed
 - The time required by the receiver object to process the message is denoted by an *activation-box*.

- **Message notation**

**return value := message(parameter:
parameterType): returnType**

s := calculateSalary(hours)

s := calculateSalary (hours: Integer): Double

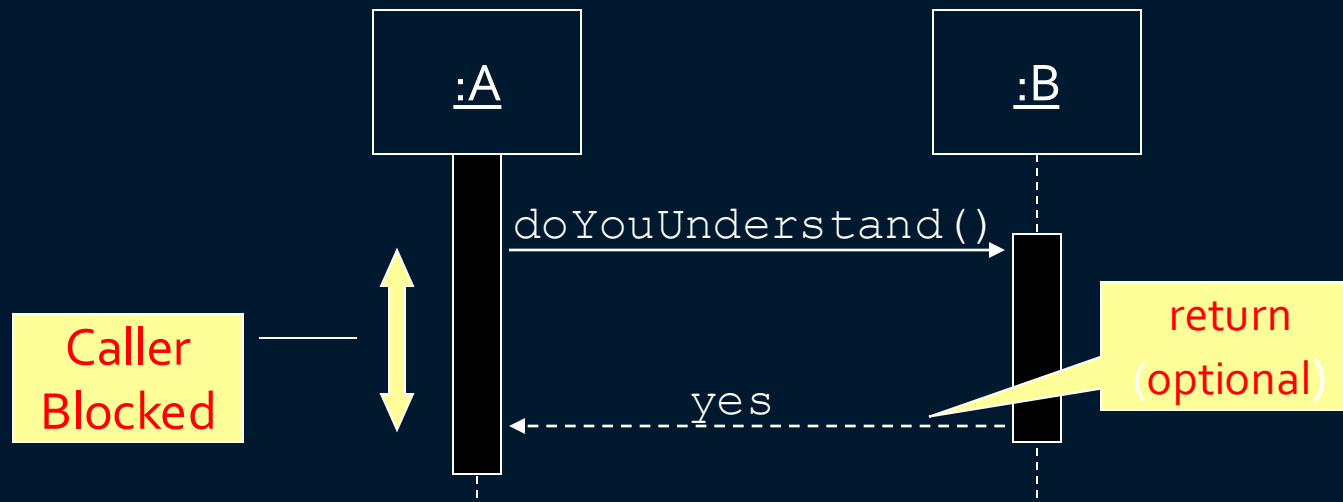
Return Values



- Optionally indicated using a dashed arrow with a label indicating the return value.
 - Don't model a return value when it is obvious what is being returned, e.g. `getTotal()`
 - Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.
 - Prefer modeling return values as part of a method invocation, e.g. `ok = isValid()`

Synchronous Messages

- Nested flow of control, typically implemented as an operation call.
 - The routine that handles the message is completed before the caller resumes execution.
 - The execution waits until the message is received and completed





Activations

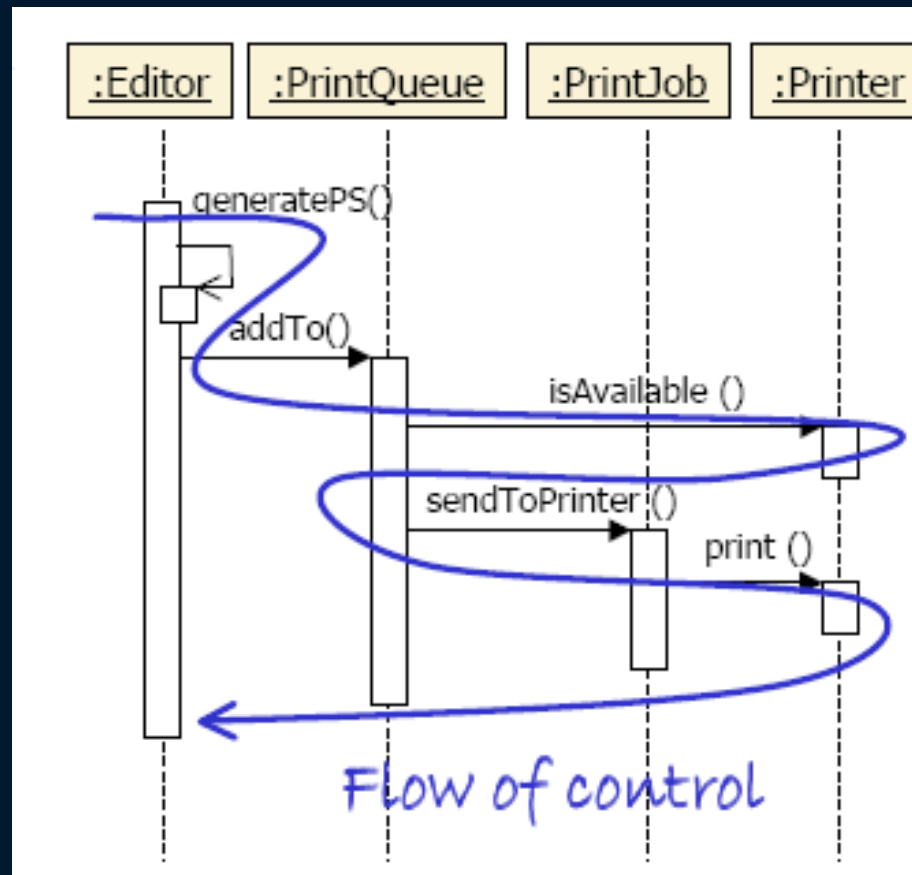
- Calling the procedure of an object activates that object. The duration of the procedure execution is called an activation.
 - In a single-thread model, only one object is activated and executing at any given moment – although other objects may be activated and waiting
 - In a multi-thread model, multiple objects may be active and executing simultaneously
- 



Activations

- An activation is shown as a white box on an object's life-line. It shows when that object is active
 - Reception of a message activates an object, which remains active while processing or waiting for nested procedures call to return.
- 

Activations





Activations

- A procedure implicitly returns control to the calling object at the end of its activation.
- Activations are useful for showing nested procedure calls : The message arrow leaves the activation rectangle and not the lifeline.

Control information

- Condition

- syntax: '[' expression ']' message-label
- The message is sent only if the condition is true
- example:

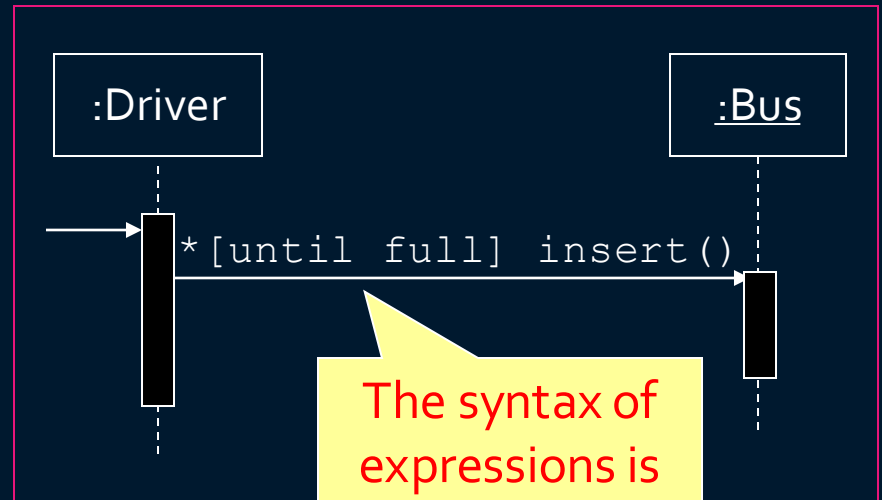
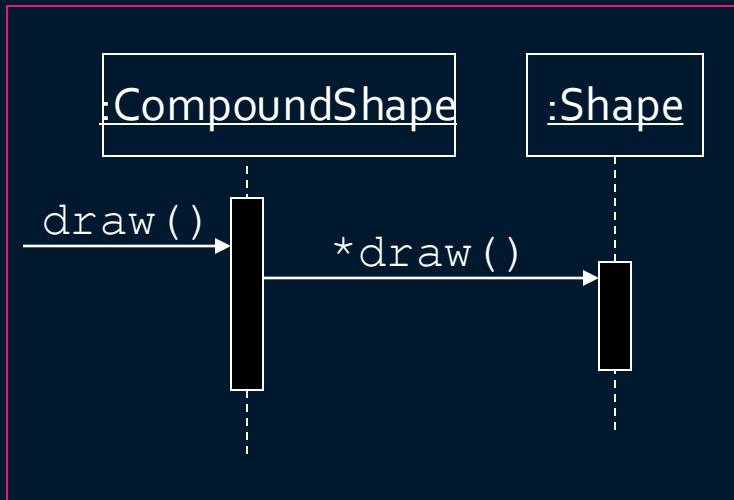
[ok] borrow(member)

- Iteration

- syntax: * ['[' expression ']'] message-label
- The message is sent many times to possibly multiple receiver objects.

Control Information

- Iteration examples:



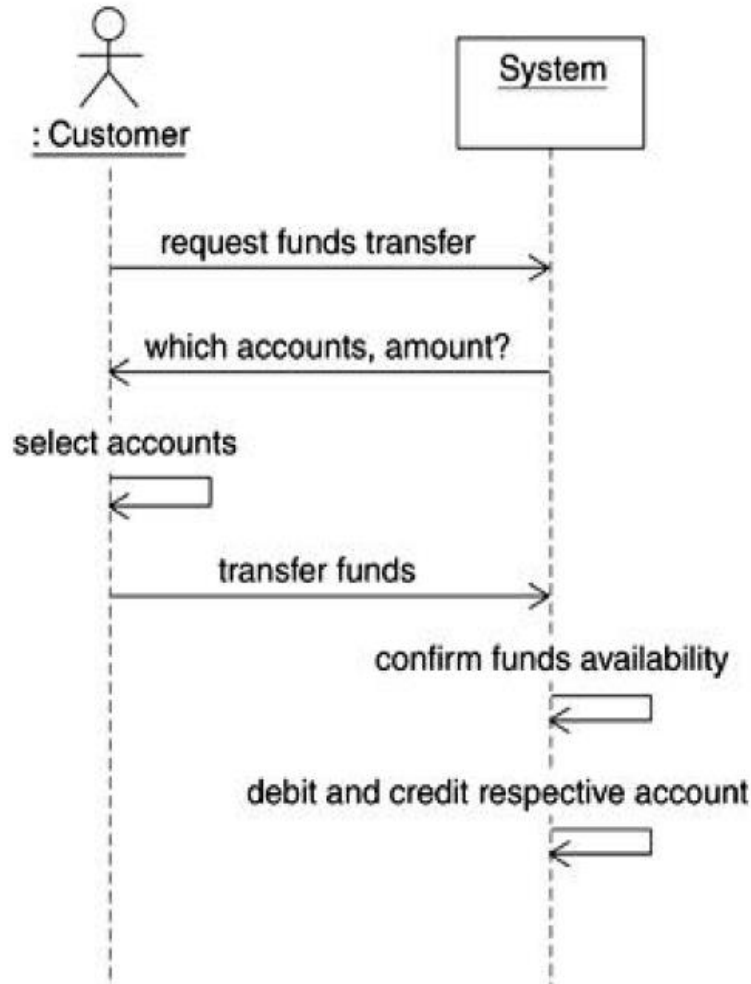
The syntax of expressions is not a standard

Control Information

- The control mechanisms of sequence diagrams suffice only for modeling simple alternatives.
 - Consider drawing several diagrams for modeling complex scenarios.
 - Don't use sequence diagrams for detailed modeling of algorithms (this is better done using *activity diagrams*, *pseudo-code* or *state-charts*).

Example

1. The customer requests a funds transfer.
2. The system asks the user to identify the accounts between which funds are to be transferred and the transfer amount.
3. Customer selects the account to transfer funds from, the account to transfer to, and then indicates the amount of funds to transfer.
4. The system checks the account from which funds are to be transferred and confirms that sufficient funds are available.
5. The amount is debited to the account from which funds are to be transferred and credited to the account previously selected by the customer.



Another Example

Session Sequence Diagram

